

TENANT-ISOLATED MEMORY

Engineering implementation note / 01 — Memory vault

I am Amin Azimi, AI Architect and developer of Aegis Agent. I designed the tenant-memory layer to make task context explicit, scoped and time-bounded. In this document I distinguish the implemented memory mechanics from stronger security properties that the source code does not yet establish.

01 | Isolation model

I organize transient entries by tenant ID → namespace → memory key. The TenantMemoryVault stores each value in an instance-local dictionary, and reads resolve the requested tenant and namespace before returning a value. A key present only under another tenant can cause a permission error rather than disclosing that tenant’s value.

02 | Retention and expiration

I provide a configurable default time-to-live of 3,600 seconds. Individual writes may override it. A read rejects an expired entry; the authorization check also removes expired entries when encountered. This is lazy expiration, not a scheduled background deletion service. A zero or negative TTL expires immediately.

03 | Value handling

I serialize values to canonical JSON and encode the resulting bytes using a repeating-key XOR transform, followed by URL-safe Base64. The key is derived from a SHA-256 hash of a configured string. This is obfuscation, not authenticated modern encryption; the default fallback key is embedded in source. I do not describe this implementation as production-grade cryptographic protection.

04 | Scope of persistence

The vault dictionary lives in process memory. I do not claim that entries survive a process restart or are synchronized across workers. The separate twin_interview component persists consented profiles in SQLite; that durability is a different mechanism and should not be confused with transient vault persistence.



MEMORY CONTROL & BOUNDARIES

05 | Operator-visible profile controls

I expose a separate twin_memory_control path that can generate a local memory.md view of profile fields and record forgotten field names in SQLite. A field-level forget operation suppresses that field from subsequent generated listings; it does not by itself erase the underlying value from the stored profile.

06 | Removal operations

I provide vault operations to delete one entry, clear a namespace, or forget entries matching a predicate. Separately, forget_all removes selected tenant-local action, feedback, session-lock and export artifacts and writes a deletion receipt. Its documented implementation does not establish universal erasure across every table, backup, process or downstream copy.

07 | Key lifecycle and storage paths

I expose a master-key fingerprint for metadata inspection and a key-rotation setter. The setter does not re-encrypt existing vault entries; rotating a key without migration can make earlier values unreadable. A hashed directory path can be created for tenant RAG storage, but path creation alone does not prove a deployed vector database or secure retrieval pipeline.

08 | Security and tenancy limits

I treat tenant-scoped dictionary lookup as an implementation boundary, not as end-to-end authorization. The vault accepts tenant IDs supplied by callers; identity binding must be enforced at the surrounding trusted boundary. A missing tenant ID normalizes to default. Snapshots expose decrypted values for the requested tenant and must be guarded by the caller.

09 | Source-backed engineering position

My source review covers core/tenant_memory.py, core/twin_memory_control.py and core/twin_interview.py. The supported claims are namespace separation, in-process TTL handling, basic deletion APIs and SQLite-backed profile controls. I do not claim independently executed penetration tests, tenant isolation across all API routes, durable vault replication or compliance certification.

DESIGN STATUS — ACTIVE DEVELOPMENT

I continue to harden identity binding, cryptography, key migration, deletion semantics and isolation tests before treating this layer as a production security boundary.